

MPLS Label Forwarding Simulation

Allowed Languages: C, C++, or Java

1. Objective

The purpose of this assignment is to understand the fundamental operation of the Multiprotocol Label Switching (MPLS) protocol. You will simulate the creation of routing tables (Routing Information Base or RIB) and label forwarding tables (Label Forwarding Information Base or LFIB) for a fixed network topology and trace the path of a labeled packet.

2. A Brief Introduction to MPLS

What is MPLS?

In traditional IP routing, every router makes an independent forwarding decision by looking at the destination IP address in a packet's header. This involves a complex lookup in a large routing table.

Multiprotocol Label Switching (MPLS) is a high-performance packet forwarding technology. Instead of complex IP lookups, it uses short, fixed-length **labels** to make forwarding decisions.

Here's the basic idea:

1. **Ingress Router (Entrance):** When a packet enters an MPLS network, the first router (the "Ingress" router) analyzes its destination IP (and other factors) and assigns it to a **Forwarding Equivalence Class (FEC)**. It then "pushes" (adds) a label to the packet.
2. **Transit Routers (Middle):** Routers inside the network ("Transit" routers) don't look at the IP address. They *only* look at the label. They use the incoming label to index a simple table (the **LFIB**) which tells them what new label to "swap" it with and which outgoing interface to send it to. This is extremely fast.
3. **Egress Router (Exit):** The last router in the path ("Egress" router) "pops" (removes) the label and forwards the original IP packet to its final destination.

This assignment simulates this exact process: you'll first build the IP-level routing (Task 2) and then use that to build the MPLS-level forwarding tables (Task 4) and simulate the packet flow (Task 5).

3. Fixed Network Topology

You must use the following fixed 4-router topology for your assignment. Routers are numbered 0, 1, 2, and 3.

- **Nodes (Routers):** R0, R1, R2, R3
- **Links (Bidirectional):**
 - R0 <-> R1 (Cost: 10)
 - R0 <-> R2 (Cost: 20)
 - R1 <-> R3 (Cost: 20)

- R2 <-> R3 (Cost: 10)

4. Tasks

Task 1: Represent the Topology

In your chosen language, create a data structure to represent the network topology.

Task 2: Compute Routing Tables (RIB)

Implement **Dijkstra's algorithm** to calculate the shortest-cost path from *each* router to *all other* routers in the network. This table represents the "intelligence" (the RIB) that each router uses to make forwarding decisions.

Output: Your program must first print the routing table for all 4 routers. The format for each router's table should be:

```
Routing Table for Router R<id>:
Destination | Next Hop | Total Cost
-----
R<dest> | R<next> | <cost>
...
```

Task 3: Define a Forwarding Equivalence Class (FEC)

For this assignment, we will focus on a single **Forwarding Equivalence Class (FEC)**:

- "All traffic originating from R0 and destined for R3."

A Note on How Labels are Assigned

In a real-world network, routers don't just know which labels to use. They are assigned automatically using a **Label Distribution Protocol (LDP)**.

The key concept is: **A label is assigned by the downstream router (the receiver) and advertised upstream (to the sender).**

Let's trace this for our path (R0 -> R2 -> R3):

1. **R3 (Egress Router):** Knows the FEC (traffic for R3) terminates here. It allocates a label (e.g., **777**) and tells its neighbor, R2: "If you have traffic for this FEC, send it to me with Label 777."
2. **R2 (Transit Router):** Receives this. It creates an LFIB entry: (In-Label: ???) -> (Out-Label: 777, Next-Hop: R3). It then allocates its *own* label from its pool (e.g., **300**) and tells its neighbor, R0: "If you have traffic for this FEC, send it to me with Label 300."
3. **R0 (Ingress Router):** Receives this. It creates its LFIB entry: (FEC: R0->R3) -> (Push 300, Next-Hop: R2).

This process builds the "Label Switched Path" (LSP) automatically.

For this assignment, you do not need to simulate LDP. You will use the *results* of this process, which are the label values given in Task 4.

Task 4: Create Label Forwarding Tables (LFIB)

Based on the shortest path for the FEC (R0 -> R3) you calculated in Task 2, you will now create the **Label Forwarding Information Base (LFIB)** for each router *along that path*.

The shortest path from R0 to R3 is **R0 -> R2 -> R3** (Total Cost: 20 + 10 = 30).

Your simulation must use the following label assignments. You must implement these as data structures (e.g., a Map, Dictionary, or a simple struct) that your simulation function can read from.

1. Ingress Router (R0):

- **Logic:** R0 sees a packet for R3. It looks up its RIB (from Task 2) and sees the next hop is R2. Because this is an MPLS simulation, it also looks up its "FEC-to-Label" table.
- **LFIB Entry (FEC -> Label Op):** (FEC: R0->R3) -> (Push 300, Send to R2)

2. Transit Router (R2):

- **Logic:** R2 receives a packet. It *ignores* the IP address and *only* looks at the label.
- **LFIB Entry (Label -> Label Op):** (In-Label: 300) -> (Swap for Out-Label: 777, Send to R3)

3. Egress Router (R3):

- **Logic:** R3 receives a packet. It looks at the label.
- **LFIB Entry (Label -> Label Op):** (In-Label: 777) -> (Pop Label, Deliver Locally)

Task 5: Simulate Packet Forwarding

Write a function or main logic block that simulates sending one packet for the defined FEC.

How to simulate:

1. Create a simple "Packet" object or struct. It should have properties like source, destination, and label. Initially, label is empty.
2. Start at R0. Your R0 function should look at the packet's destination (R3), find the matching FEC, and consult its LFIB (from Task 4).
3. Based on the LFIB, it should "push" the label (set packet.label = 300) and print its action.
4. Call the function for R2, passing the modified packet.
5. Your R2 function should *only* look at packet.label. It consults its LFIB, "swaps" the label (set packet.label = 777), and prints its action.
6. Call the function for R3.
7. Your R3 function looks at the label, "pops" it (set packet.label = empty), and prints the final delivery message.

Simulation Output: The console output for this part must look *exactly* like this:

Simulating packet from R0 to R3...

[R0] Packet for R3 (FEC: R0->R3). Pushing Label 300. Sending to R2.

[R2] Received packet with In-Label 300. Swapping for Out-Label 777. Sending to R3.

[R3] Received packet with In-Label 777. Popping label. Packet delivered.

5. Submission Requirements

1. **Source Code:** Your complete, well-commented source code file(s) (.c, .cpp, or .java).
2. **README File:** A README.txt file that includes:
 - Your Name and ID.
 - Clear instructions on how to compile and run your program.
 - Your answers to the "Analysis Questions" below.
3. **Output File:** A simple text file named output.txt that contains the *complete and final console output* of your program, showing both the routing tables from Task 2 and the packet simulation trace from Task 5.